REVIEW 1: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 1: Introduction

1.1 Project Overview

The evolution of Command Line Interfaces (CLIs)

Command-line interfaces have been the main way to control Unix and Linux systems since the first UNIX shell, and shells such as Bash and Zsh are still what most administrators use every day. They are fast, they can be scripted, and they give direct control over files, processes and storage. The catch is that the user has to remember the exact command, its flags and the order of its arguments.

The shift toward Natural Language Processing (NLP) in system administration

Large Language Models have changed what a terminal can look like. Instead of remembering flags, a user can describe what they want, and an agent turns that into real system operations. Our class is building one such "Agentic Shell": a base shell sends each request to one of 35 specialised agents. Our part is Agent #12, the Smart Archive Agent, which handles archiving, compression, extraction, inspection and encryption on Ubuntu Linux.

1.2 Problem Statement

Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Linux has several archive and compression tools (tar, gzip, bzip2, xz, zip and gpg), and each one has its own flags. An xz tarball needs tar -cJf, gzip needs -czf and bzip2 needs -cjf. xz and gzip delete the original file unless you remember -c, and encrypting with GPG adds --symmetric and --cipher-algo AES256. Unless you use these tools every day, you end up looking the flags up every time.

Risk of executing incorrect commands due to human error

Mistakes here can lose data. If the source and destination are swapped in a tar command, tar can overwrite the source file with an empty archive; we reproduced this ourselves (Appendix A, Figure A.2). Extracting an untrusted archive can also write files outside the target folder (the "Zip Slip" problem) or overwrite existing files. Our agent avoids this by only calling a fixed set of functions that check every path first and never overwrite anything.

1.3 Objectives

- **Lightweight NL-to-CLI translation:** turn plain-English requests into archive operations without heavy libraries or an internet connection.
- **Predefined operation boundaries:** map every request to one of eight fixed tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) instead of running shell commands typed by the user.
- **MCP-style tool design:** describe each tool with a name, a description and a JSON Schema, the way the Model Context Protocol describes tools, so the class shell's LLM can call our agent.
- **Security and guardrails:** check every path, block Zip Slip, never overwrite files, and provide AES-256 GPG encryption for sensitive files.
- **Structured feedback:** always return a clear result: what was created, what is inside an archive, or what went wrong.

1.4 Scope and Limitations

- **Supported operating systems:** Ubuntu Linux, including Ubuntu under WSL 2 on Windows, which is what we develop on. The agent uses the standard tools tar, gzip, bzip2, xz, file and gpg, and Python (written for 3.8 or newer, tested on 3.14); zip files are handled by Python's built-in zipfile module.
- **Terminal environments:** it runs in a normal terminal (Bash or Zsh), either as an interactive prompt or as a one-line command.
- **Administrative privileges:** it runs as a normal user and never asks for sudo, so it cannot damage system files.
- **Domain boundaries:** it only handles archiving, compression and encryption. Anything else, such as process management or network settings, is rejected so that the shell can send it to the right agent.

Chapter 2: Literature Review & Theoretical Background

2.1 Large Language Models (LLMs) in Code Generation

Capabilities of modern LLMs in structured text and code synthesis

Current LLM families such as OpenAI GPT, Anthropic Claude and Google Gemini are good at turning plain sentences into structured output. With function calling, a model fills in a predefined JSON schema instead of writing free text, which is exactly what an agent needs to pick a tool and its arguments.

Challenges with LLM hallucination in system-level execution

The problem is that LLMs can hallucinate: they may invent file paths, make up flags or pick a destructive command. Running a model's output directly in bash is therefore unsafe. That is why we kept understanding the request separate from running it: whatever produces the tool call, every argument is checked by our own validation code before anything touches the system.

2.2 Introduction to Model Context Protocol (MCP)

Architecture of MCP (Clients, Servers, and Tools)

The Model Context Protocol (MCP), published by Anthropic, is an open standard for connecting AI applications to tools and data. It has three parts: clients (the application that uses tools), servers (programs that offer tools) and tools (single functions, each described with a JSON Schema for its inputs).

Why MCP is preferred over traditional rigid API tooling for local system access

Before MCP, connecting a model to local tools usually meant reading command output as plain text or writing a custom adapter for every tool. MCP gives one standard way to discover tools, check their inputs and report errors. Because we describe our tools in the same way, our agent works on its own today and can be plugged into the class-wide shell without changing its archive and security code.

2.3 Existing Solutions

We compared our approach with the terminal assistants that already exist:

Tool	How it works	Safety	What it means for our project
Plain Bash	The user types every command by hand	Only normal user permissions; nothing checks the syntax	The tools themselves are reliable, but you must remember every flag
Warp AI	Terminal with AI command suggestions	Suggests shell commands; the user has to review them before running	Shows that plain-English terminals work, but it still runs whatever shell script it suggests
GitHub Copilot CLI	Turns a sentence into a shell command	Explains the command and asks the user to confirm	Shows LLM command generation, but the model can still make up flags
Smart Archive Agent (ours)	One agent focused only on archives	Fixed tools only; every path checked; Zip Slip blocked; never uses root	Plain-English requests, but every action goes through our own checks

References

[1] Anthropic, 'Model Context Protocol (MCP) Specification,' 2024. [Online]. Available: https://modelcontextprotocol.io/

[2] OpenAI, 'Function Calling and Structured Outputs Guide,' 2024. [Online]. Available: https://platform.openai.com/docs/guides/function-calling

[3] Free Software Foundation, 'GNU Tar Manual: Tape Archiving Utility,' GNU Project, 2023. [Online]. Available: https://www.gnu.org/software/tar/manual/

[4] Warp Technologies, 'Warp: The AI-Powered Terminal User Guide,' 2024. [Online]. Available: https://docs.warp.dev/

[5] GitHub Inc., 'GitHub Copilot in the CLI Documentation,' 2024. [Online]. Available: https://docs.github.com/en/copilot/using-github-copilot-in-the-cli

Appendix A: Evidence of Work — Review 1 Stage

This appendix shows what we did in the first stage of the project: setting up our environment and checking the problem for ourselves before writing the code. Everything below is real output from our machine, and the full source code is public on GitHub (see "GitHub Repository and Project Structure" at the end of this document).

A.1 Our Development Environment

We develop on Ubuntu running inside WSL 2 on Windows, using the archive and encryption tools that come with Ubuntu. These are the versions we used:



Figure A.1: Operating system, kernel, Python and tool versions on our development machine.

A.2 Checking the Problem with the Raw Linux Tools

Before writing any code, we wanted to see the problem from Section 1.2 for ourselves, so we tried the common mistakes in a test folder:

- **Swapping tar's arguments loses data:** report.txt held some text. We ran tar -cJf report.txt backup.tar.xz, with the source and destination the wrong way round. tar printed an error, but by then it had already replaced report.txt with an empty xz archive, so the text was gone.
- **xz removes the original:** after xz notes.txt, only notes.txt.xz is left.
- **The correct manual steps are long:** doing the brief's task by hand (archive heavily, encrypt, delete the plaintext) took us three commands and eight flags (XZ_OPT=-9e, -cJf, --batch, --pinentry-mode, --passphrase, --symmetric, --cipher-algo AES256, rm). With our agent it is one sentence.

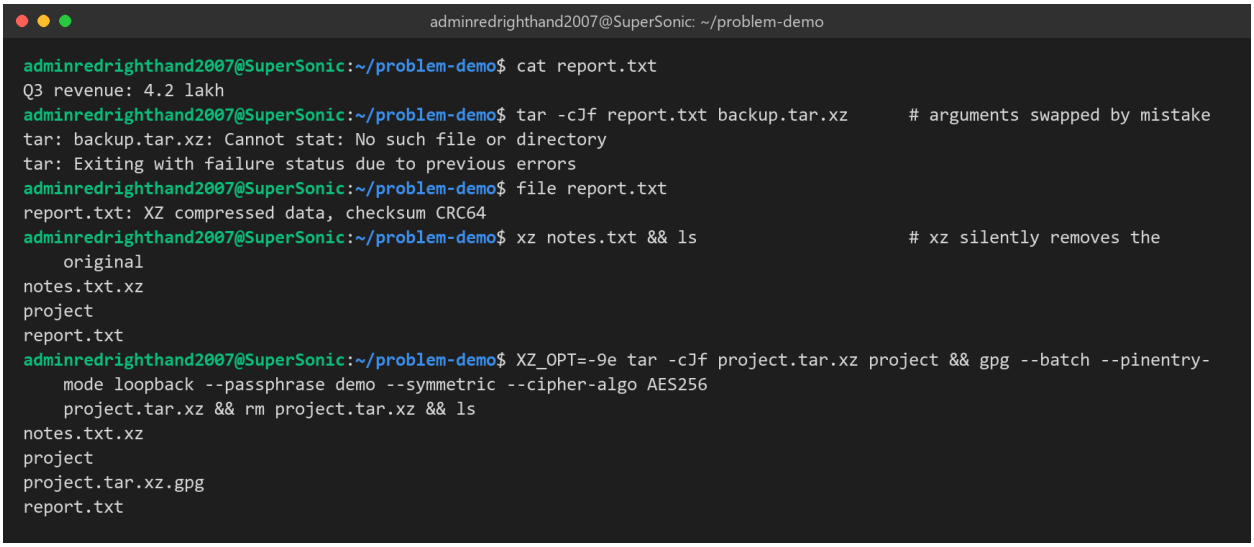


Figure A.2: Data lost by swapped tar arguments, xz deleting its input, and the manual commands needed for the brief's task.

REVIEW 2: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 3: System Architecture & Design

3.1 High-Level Architecture

The User-to-Shell Loop: User Input → LLM Processing → Tool Call → System Execution → Feedback

Our agent is Agent #12 of the class Agentic Shell. In the full shell, the base shell's LLM reads what the user typed, picks one of our tools and fills in its arguments, and our code does the actual work on the system. We also wanted the agent to work on its own, without any LLM, so we wrote a rule-based planner that does the "LLM processing" step itself and produces the same tool calls. Either way, every call reaches the system through one checked entry point, execute_tool().

Loop stage	Inside the Agentic Shell	When our agent runs on its own
User Input	The user types into the base shell.	The user types at our You: prompt (main.py).
LLM Processing	The shell's LLM reads our tool definitions (get_tools()) and picks tools by function calling.	intent.py and planner.py turn the sentence into a plan of tool calls, offline.
Tool Call	The shell calls execute_tool(name, arguments).	Each step of our plan goes to execute_tool().
System Execution	tar, gzip, bzip2, xz and gpg run through subprocess.run() with argument lists.	Same.
Feedback	A result dictionary goes back to the LLM as the tool result.	We print the result: done lines, the output path, archive entries or the error.

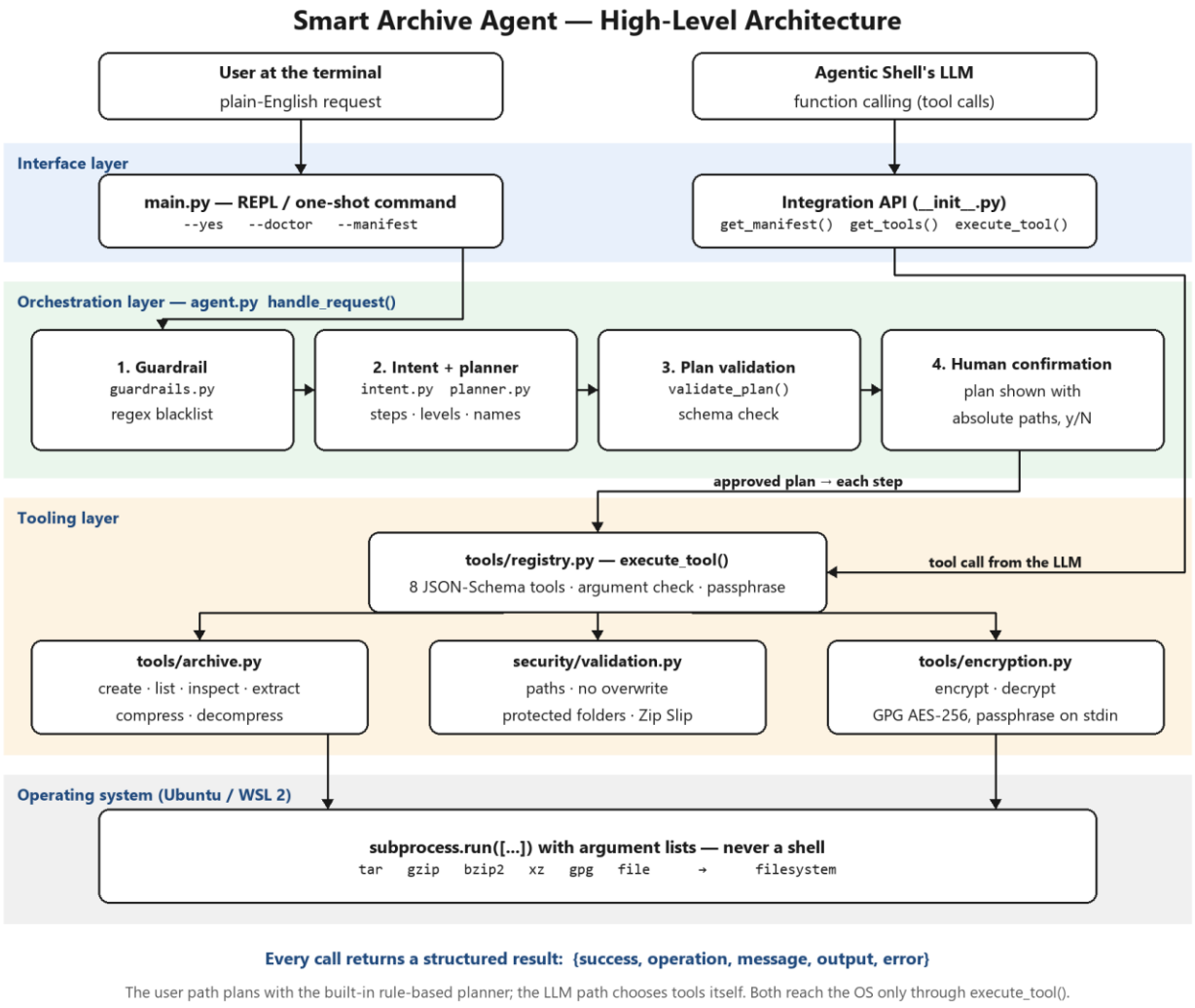


Figure 3.1: High-Level System Architecture of the Smart Archive Agent.

How to read Figure 3.1. There are two ways in, shown at the top: a user typing in the terminal, and the Agentic Shell's LLM. A typed request goes through our orchestration layer: the guardrail, the intent recognizer and planner, the plan check and the user's confirmation. A tool call from the LLM skips planning, because the LLM has already chosen the tool, and goes straight to the tool registry. Both paths then pass the same checks: execute_tool() compares the call with its JSON Schema, the security module checks every path again, and only then do our archive and encryption modules run the Linux tools. Every call returns the same kind of result.

3.2 Component Design

3.2.1 The Interface Layer: Custom REPL (Read-Eval-Print Loop) Environment

The interface is main.py together with the run() loop in agent.py. In interactive mode it prints a short banner and a You: prompt, shows the plan with full paths, asks y/N, asks for the passphrase without

showing it, and prints the result. We also added a one-shot mode for scripts. Ctrl+C or the end of input exits cleanly, and the exit code is 0 for success and 1 for failure.

```
python3 -m smart_archive_agent                          # interactive REPL
python3 -m smart_archive_agent "archive ./src"          # one request, asks before writing
python3 -m smart_archive_agent --yes "back up ./src"    # one request, no confirmation
python3 -m smart_archive_agent --doctor                 # checks tar, gzip, bzip2, xz, gpg,
file
python3 -m smart_archive_agent --manifest               # agent description for the shell's
router
```

Because our teacher will plug the agent into the class shell, the package itself is also part of the interface: __init__.py exports get_manifest(), get_tools(), execute_tool() and handle_request(), and agent.json tells the shell's router what our agent handles, what it does not, and what it needs installed.

3.2.2 The Orchestration Layer: Intent Routing

The template mentions LangChain, LlamaIndex or a raw MCP client for this layer. We decided to write the orchestration ourselves in plain Python (agent.py, handle_request()) instead of using a framework. We had three reasons: the agent has to install on our teacher's machine without extra packages, we wanted every decision to be predictable so we could reason about safety, and understanding open-ended language is already the job of the base shell's LLM. Each request goes through these stages in order:

Stage	Module	What it does
1. Guardrail	security/guardrails.py	Checks the raw text against a regex blacklist of destructive commands before anything else.
2. Intent recognition	intent.py	Removes file paths from the text, then looks for whole-word verbs in a fixed order (decrypt, encrypt, extract, decompress, list, compress, archive). Requests that are not about archiving stop here.
3. Planning	planner.py	Builds the steps: folders are archived first; "heavily" → level max, "quickly" → fast; missing output names are chosen next to the source; an unencrypted archive we created is deleted after encryption.
4. Plan validation	planner.validate_plan()	Checks every step against the tool schemas; a clean-up step may only delete a file made earlier in the same plan.
5. Confirmation and execution	agent.run_plan()	Shows the plan, waits for y, runs the steps in order, and stops at the first failure, saying which steps finished.

3.2.3 The Tooling Layer: Safe System Functions

Our agent offers eight archive functions as tools. In tools/registry.py each tool has a name, a description written so an LLM can choose it, and a JSON Schema for its parameters; this is the same shape an MCP

server uses to describe its tools. get_tools() returns these definitions in generic, OpenAI-compatible or Anthropic format, and execute_tool() runs one call. We have not wrapped them in an actual MCP server yet; we list that as future work. System monitoring, network configuration and other file tasks belong to other agents in the class shell.

Tool	Python function	Linux backend	Key arguments
create_archive	archive.create_archive()	tar with gzip / bzip2 / xz (tar -I), or Python zipfile	source_path, output_path, compression, level
list_archive	archive.list_archive()	tar -tf, zipfile	archive_path
inspect_archive	archive.inspect_archive()	file -b, tar -tf, zipfile	archive_path
extract_archive	archive.extract_archive()	Python tarfile / zipfile after checking every member	archive_path, destination_path
compress_file	archive.compress_file()	gzip, bzip2, xz	source_path, output_path, algorithm, level
decompress_file	archive.decompress_file()	gzip -d, bzip2 -d, xz -d	source_path, output_path
encrypt_file	encryption.encrypt_file()	gpg --symmetric --cipher-algo AES256	source_path, output_path (+ passphrase)
decrypt_file	encryption.decrypt_file()	gpg --decrypt	source_path, output_path (+ passphrase)

Compression levels: fast = gzip -1 / xz -1, normal = gzip -6 / xz -6, max = gzip -9 / bzip2 -9 / xz -9e.

3.2.4 Result Handling & Diagnostics

Every tool and every request returns one dictionary: success (true or false), operation, a message for people, output (the path that was created), plus entries for listings, steps and plan for multi-step requests, missing when a detail could not be found, and error when something failed. execute_tool() never raises an exception; errors inside a tool come back as success = false with a message, so a failing operation cannot crash our REPL or the base shell. The same dictionary can be passed back to an LLM with json.dumps().

3.3 Security & Guardrail Framework

3.3.1 The "Human-in-the-Loop" Validation Mechanism

Nothing is written until the user has seen and approved the whole plan. Before any step that writes a file, our agent prints every step with full source and destination paths and waits for y; any other answer cancels with "Nothing was changed". Read-only requests (list, inspect) run without asking. The passphrase for encryption and decryption is always typed by the user at a hidden prompt (twice when encrypting), never supplied by an LLM. When the agent runs inside the base shell, handle_request(confirm=…, passphrase=…) lets the shell show the plan and collect the passphrase in its own way.

3.3.2 Command Sanitization and Regex-Based Blacklisting of Destructive Commands

User text is never run as a command. We start every tool with subprocess.run() and a list of arguments (never shell=True), so characters like ; | $( ) are treated as plain text. On top of that, we check the raw request against a regex blacklist before planning, so a destructive request is refused completely instead of being half carried out:

Pattern (examples)	Refused as
rm -rf …, rm -fr …	recursive forced delete
mkfs, mkfs.ext4	filesystem formatting
dd if=…	raw disk write
:(){ :|:& };:	fork bomb
> /dev/sda	raw block-device overwrite
chmod -R 777 /	world-writable root filesystem
shutdown, reboot, poweroff, halt	system power control
format / wipe / erase … disk, drive, everything	disk wipe request
delete everything, delete all files	mass deletion request
; rm, | bash, `…`, $(…)	shell command injection
a bare / as a path	operation on the whole root filesystem

Inside every tool there are more checks, whichever way the call arrived:

- **Protected locations:** we never write into /bin, /boot, /dev, /etc, /lib, /lib64, /proc, /sbin, /sys, /usr or /var/lib.
- **No overwrites:** an existing file is never replaced; source and output must be different; an archive cannot be written inside the folder being archived.
- **Zip Slip prevention:** every member of an archive must stay inside the destination; symbolic and hard links inside tar archives are refused; the destination folder is only created after the whole archive has passed.
- **Schema check of tool calls:** unknown tools, missing or extra arguments, wrong types and invalid values (for example level = "ultra") are rejected before anything runs.
- **Least privilege:** the agent runs as a normal user and never uses sudo.

Chapter 4: Implementation Methodology

4.1 Development Environment & Tech Stack

Item	What we used
Programming Language	Python. We wrote the code for Python 3.8 or newer using only the standard library (subprocess, tarfile, zipfile, getpass, json, re, pathlib); we developed and tested it with Python 3.14.4.
Operating System	Ubuntu 26.04.1 LTS on Windows Subsystem for Linux 2 (Windows 11). It is meant for any Ubuntu or Debian machine.
LLM Provider	Provided by the base Agentic Shell, which owns the LLM. get_tools() gives our tool definitions in OpenAI-compatible format (which local Ollama also accepts) and Anthropic Claude format. Our agent does not call an LLM itself and needs no API key; offline requests use our own planner.
SDKs	None needed. We did not use the MCP Python SDK; our tool definitions follow the same JSON Schema structure, so we can add an MCP server later without changing the tools.
System Utilities	GNU tar 1.35, gzip, bzip2, XZ Utils 5.8.3, GnuPG 2.4.8, file.
Testing	pytest 9.1.1 (58 automated tests) and our script benchmarks/evaluate.py for accuracy, speed and error recovery.
Packaging & Integration	pyproject.toml (pip install from GitHub, smart-archive-agent command), the agent.json manifest, INTEGRATION.md and a runnable integration example.
Tools	Visual Studio Code with the WSL extension, Git and GitHub.

4.2 Step-by-Step Implementation

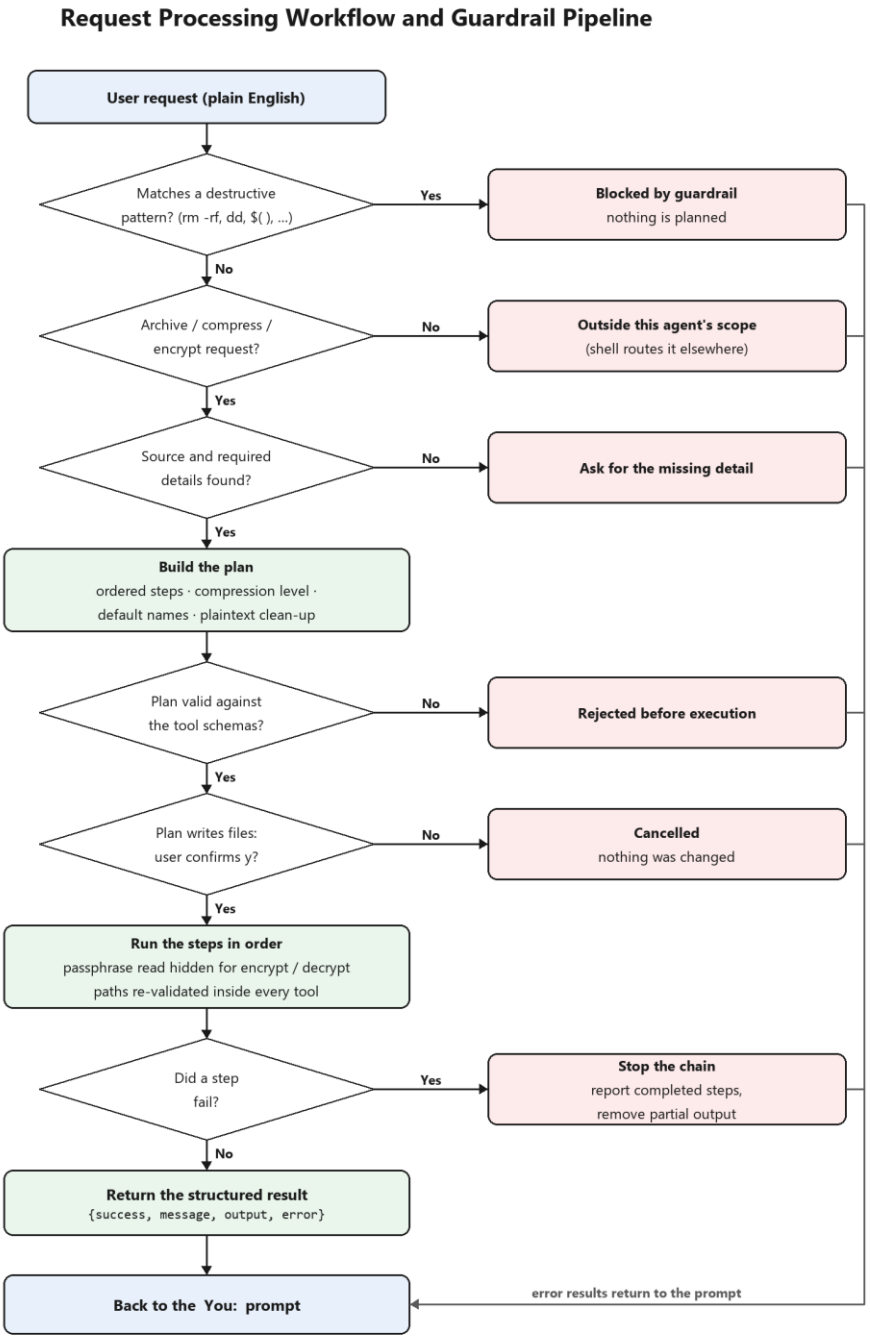


Figure 4.1: End-to-End Request Processing Workflow and Guardrail Pipeline.

Figure 4.1 follows one request through our agent. Every decision has a safe way out on the right (blocked, out of scope, missing detail, rejected, cancelled, or a failed step), which returns an error and goes back to the prompt without changing any file. We built the implementation in four steps.

Step 1: Setting up the base CLI prompt loop

We started with main.py, which reads the command line (interactive, one-shot, --yes, --doctor, --manifest, --version), and the run() loop in agent.py: read a line, pass it to handle_request() with a terminal confirmation and a hidden passphrase prompt, print the result with print_result(), and repeat until exit, Ctrl+C or the end of input. We added --doctor so that on any lab machine we can see at once which Linux tool is missing and the apt install command for it.

Step 2: Building the MCP-style tool layer to expose safe system functions

Next we wrote the eight operations in archive.py and encryption.py as plain Python functions that check their own paths and return result dictionaries. Then, in tools/registry.py, we described each one for function calling (TOOL_SCHEMAS) and wrote execute_tool(), the only function that actually runs a tool. The template's examples, list_directory and get_system_stats, match list_archive and inspect_archive in our domain: they read an archive without changing anything. We also added an option to prefix our tool names (smart_archive__create_archive) so they stay unique among the 35 agents.

Step 3: Prompt engineering for exact tool matching and JSON format enforcement

For the LLM path we wrote each tool and parameter description so that a model can pick the right tool and values from everyday words, and we used JSON Schema to force the format:

```
"level": {"type": "string", "enum": ["fast", "normal", "max"],
          "description": "Compression effort. Use 'max' when the user says heavily, maximum
                          or as small as possible; 'fast' for quick; otherwise 'normal'."}
"required": ["source_path", "output_path"],   "additionalProperties": false
```

check_arguments() applies the schema to every call (unknown tool, missing or extra argument, wrong type, value outside the enum), and execute_tool() also accepts the JSON string that LLM APIs return. For our offline planner, intent.py matches whole words in a fixed order after removing the file paths from the text, so "zip" is not found inside "gzip" and "tar" is not found inside "backup.tar.xz". We measured how well this works on phrasings we never tuned on (Review 3, Section 5.2.1).

Step 4: Implementing the command execution wrapper and output formatter

All Linux tools run through one wrapper that passes an argument list, captures the output, uses a 600-second timeout, closes standard input and turns failures into readable errors:

```
subprocess.run(["tar", "-I", "xz -9e -T0", "-cf", output, "-C", parent, name],
               check=True, capture_output=True, text=True,
               timeout=600, stdin=subprocess.DEVNULL)

# passphrase is sent on standard input, never as an argument
gpg --batch --yes --no-symkey-cache --pinentry-mode loopback \
    --passphrase-fd 0 --symmetric --cipher-algo AES256 \
    --output <file>.gpg <file>
```

run_plan() runs the steps in order, remembers which files each step created (so the clean-up step can only delete those), stops at the first failure and removes any half-written output. print_result() prints the result in the terminal as done lines, the output path, archive entries and errors. We report how we tested all of this, and the results, in Review 3 (Chapter 5).

References

[1] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[2] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] JSON Schema, "JSON Schema specification." Available: https://json-schema.org/

[5] OpenAI, "Function calling guide." Available: https://platform.openai.com/docs/guides/function-calling

[6] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Python Software Foundation, PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

Appendix B: Implementation Evidence — Review 2 Stage

This appendix shows the implementation from Chapters 3 and 4 as it is in our repository: the size of each module, the most important parts of the code, and real output from running it.

B.1 Size of Each Module

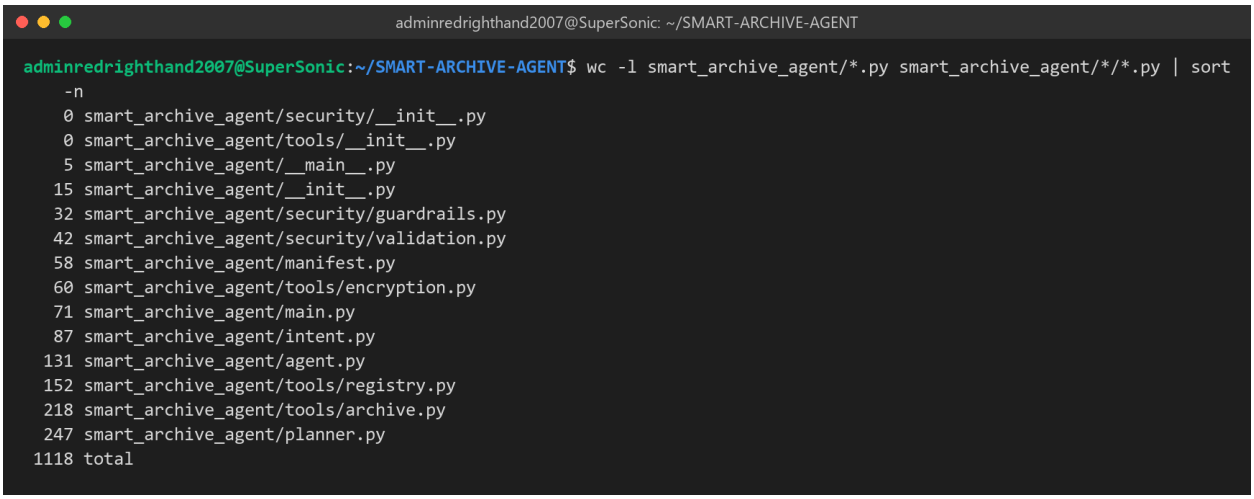


Figure B.1: Lines of code in each module of the smart_archive_agent package (1,118 lines in total).

B.2 Key Parts of Our Code

These excerpts are copied straight from our repository, with their real line numbers.

Guardrail: regex blacklist checked before any planning (Section 3.3.2)

```
  6  DESTRUCTIVE_PATTERNS = [
  7      (re.compile(r"\brm\s+-[a-z]*r[a-z]*f|\brm\s+-[a-z]*f[a-z]*r", re.I), "recursive forced delete (rm -
rf)"),
  8      (re.compile(r"\bmkfs(\.\w+)?\b", re.I), "filesystem formatting (mkfs)"),
  9      (re.compile(r"\bdd\s+if=", re.I), "raw disk write (dd)"),
 10      (re.compile(r":\(\)\s*\{\s*:\|:&\s*\};:"), "fork bomb"),
 11      (re.compile(r">\s*/dev/(sd|nvme|hd)\w*", re.I), "raw block-device overwrite"),
 12      (re.compile(r"\bchmod\s+(-R\s+)?777\s+/(\s|$)", re.I), "world-writable root filesystem"),
 13      (re.compile(r"\b(shutdown|reboot|poweroff|halt)\b", re.I), "system power control"),
 14      (re.compile(r"\b(format|wipe|erase|destroy)\b.*\b(drive|disk|partition|system|everything)\b", re.I),
 15       "disk wipe request"),
 16      (re.compile(r"\bdelete\s+(everything|all\s+files|all\s+my\s+files)\b", re.I), "mass deletion
request"),
 17      (re.compile(r"(?:^|\s)/\*?(?:\s|$)"), "operation on the whole root filesystem (/)"),
 18      (re.compile(r"[;&|`]\s*(rm|sudo|curl|wget|bash|sh)\b|\$\(", re.I), "shell command injection"),
 19  ]
 20
 21  # Output locations the agent will never write into, even if the user asks.
 22  # /root is deliberately absent: it is the home folder when the shell runs as root.
 23  PROTECTED_DIRS = ("/bin", "/boot", "/dev", "/etc", "/lib", "/lib64", "/proc",
 24                    "/sbin", "/sys", "/usr", "/var/lib")
 25
 26
 27  def check_request(text: str) -> str | None:
 28      """Return a refusal reason if the request matches a destructive pattern."""
 29      for pattern, reason in DESTRUCTIVE_PATTERNS:
 30          if pattern.search(text):
 31              return reason
 32      return None
```

Listing B.1: Our destructive-command patterns and check_request(). Source: smart_archive_agent/security/guardrails.py, lines 6–32.

Planner: chaining the archive, compression level and encryption (Section 3.2.2)

```
166      wants_encrypt = ENCRYPT_RE.search(words) is not None
167      named_archive = named_out is not None and archive_format_for(named_out) is not None
168      wants_archive = (op == "create_archive" or source_abs.is_dir() or named_archive
169                       or PACKAGE_RE.search(words) is not None)
170      wants_compress = COMPRESS_RE.search(words) is not None
171
172      steps: list[Step] = []
173      packed = None
174      if wants_archive:
175          fmt = (archive_format_for(named_out) if named_archive else None) or               ("zip" if
ZIP_FORMAT_RE.search(words) else algorithm or ("gz" if level == "fast" else "xz"))
176          packed = named_out if named_archive else str(_unique(source_abs.parent / (source_abs.name +
EXTENSIONS[fmt])))
177          steps.append(Step("create_archive", {"source_path": source, "output_path": packed,
178                                               "compression": fmt, "level": level},
179                            f"bundle into one {fmt} archive ({level} compression)"))
180      elif wants_compress:
181          algo = algorithm or ("gz" if level == "fast" else "xz")
182          packed = named_out or str(_unique(Path(str(source_abs) + "." + algo)))
183          steps.append(Step("compress_file", {"source_path": source, "output_path": packed,
184                                              "algorithm": algo, "level": level},
185                            f"compress with {algo} ({level} level)"))
186      if wants_encrypt:
187          enc_source = packed or source
188          enc_out = gpg_out or (named_out if packed is None else None) or
str(_unique(Path(str(_abs(enc_source)) + ".gpg")))
189          steps.append(Step("encrypt_file", {"source_path": enc_source, "output_path": enc_out},
190                            "encrypt with GPG AES-256"))
```

Listing B.2: How one sentence becomes an archive step and an encryption step. Source: smart_archive_agent/planner.py, lines 166–190.

Tool layer: the one checked entry point (Section 3.2.3)

```
def execute_tool(name: str, args: dict, passphrase: str | None = None) -> dict:
    """Run one tool call. Never raises: failures come back as {'success': False, ...}.

    `name` may carry the TOOL_PREFIX; `args` may be a dict or the JSON string an LLM API returns.
    """
    if isinstance(name, str) and name.startswith(TOOL_PREFIX):
        name = name[len(TOOL_PREFIX):]
    if isinstance(args, str):
        try:
            args = json.loads(args)
        except ValueError:
            return {"success": False, "operation": name, "error": "arguments are not valid JSON",
                    "message": "The tool call was rejected before execution."}
    problems = check_arguments(name, args)
    if problems:
        return {"success": False, "operation": name, "error": "; ".join(problems),
                "message": "The tool call was rejected before execution."}
    try:
        kwargs = dict(args)
        if name in NEEDS_PASSPHRASE:
            kwargs["passphrase"] = passphrase or ask_passphrase(name)
        return FUNCTIONS[name](**kwargs)
    except Exception as exc:
        return {"success": False, "operation": name, "error": str(exc),
                "message": "The operation could not be completed. Review the error and try again."}
```

Listing B.3: execute_tool() checks every call and never raises an exception. Source: smart_archive_agent/tools/registry.py, lines 128–152.

Zip Slip check before anything is written (Section 3.3.2)

```
    root = destination.resolve()

    def check(name: str) -> None:
        target = (root / name).resolve()
        if target != root and root not in target.parents:
            raise ValidationError("Archive contains a path that escapes the destination.")

    # Every member is checked before the destination is created or anything is written.
    if zipfile.is_zipfile(archive):
        with zipfile.ZipFile(archive) as zf:
            for name in zf.namelist():
                check(name)
            destination.mkdir(exist_ok=True)
            zf.extractall(destination)
    else:
        with tarfile.open(archive, "r:*") as tf:
            for member in tf.getmembers():
                check(member.name)
                if member.issym() or member.islnk():
                    raise ValidationError("Archive contains links; extraction stopped for safety.")
```

Listing B.4: Every archive member is checked before the destination folder is created. Source: smart_archive_agent/tools/archive.py, lines 139–158.

Passphrase handling for GPG (Section 4.2, Step 4)

```
 8  GPG_BASE = ["gpg", "--batch", "--yes", "--no-symkey-cache", "--pinentry-mode", "loopback",
 9              "--passphrase-fd", "0"]
10
11  def _gpg(args: list[str], passphrase: str, failure: str) -> None:
12      try:
13          subprocess.run(GPG_BASE + args, input=passphrase + "\n", check=True,
14                         capture_output=True, text=True, timeout=600)
15      except FileNotFoundError as exc:
16          raise RuntimeError("GPG is not installed. Install it with: sudo apt install gnupg") from exc
17      except subprocess.CalledProcessError as exc:
18          lines = (exc.stderr or exc.stdout or "").strip().splitlines()
19          detail = lines[-1].replace("gpg: ", "", 1) if lines else ""  # Python 3.8: no removeprefix
20          if "bad session key" in detail.lower() or "bad passphrase" in detail.lower():
21              detail = "wrong passphrase"
22          raise RuntimeError(f"{failure} ({detail})" if detail else failure) from exc
```

Listing B.5: The passphrase reaches gpg on standard input, never as an argument. Source: smart_archive_agent/tools/encryption.py, lines 8–22.

B.3 A Tool Definition as the LLM Sees It

This is what get_tools("openai", prefixed=True) returns for our first tool, exactly what the Agentic Shell's LLM receives (Section 4.2, Step 3):

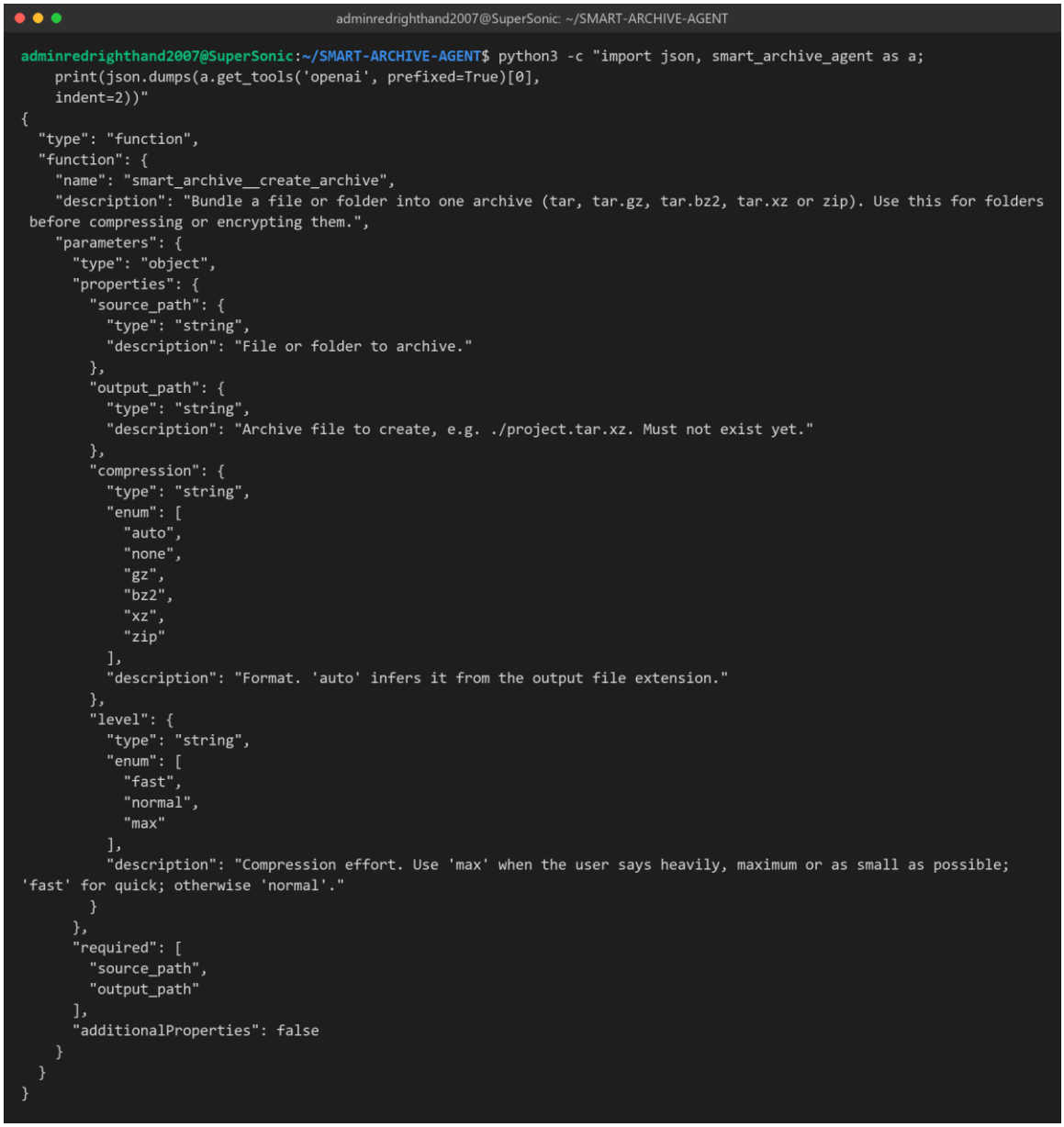


Figure B.2: Our create_archive tool definition in OpenAI-compatible function-calling format.

B.4 The Plan Our Agent Builds for the Brief's Request

Here we only asked the planner for a plan, without running it. The brief's sentence becomes three steps at level max, and the encryption step uses the archive step's output as its input.

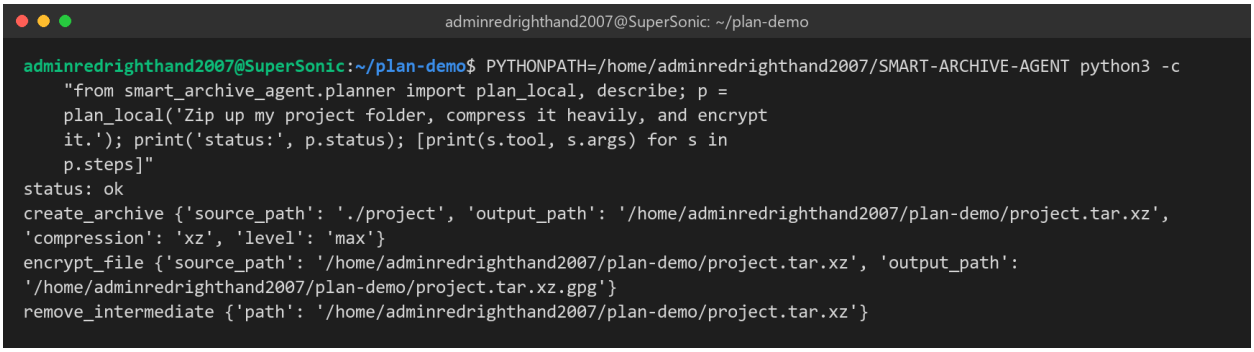


Figure B.3: plan_local() output for "Zip up my project folder, compress it heavily, and encrypt it."

B.5 Running Our Integration Example

examples/agentic_shell_example.py acts like the base shell: it loads our agent, offers its tools, routes the request, runs two LLM-style tool calls, and then tries the offline planner path:

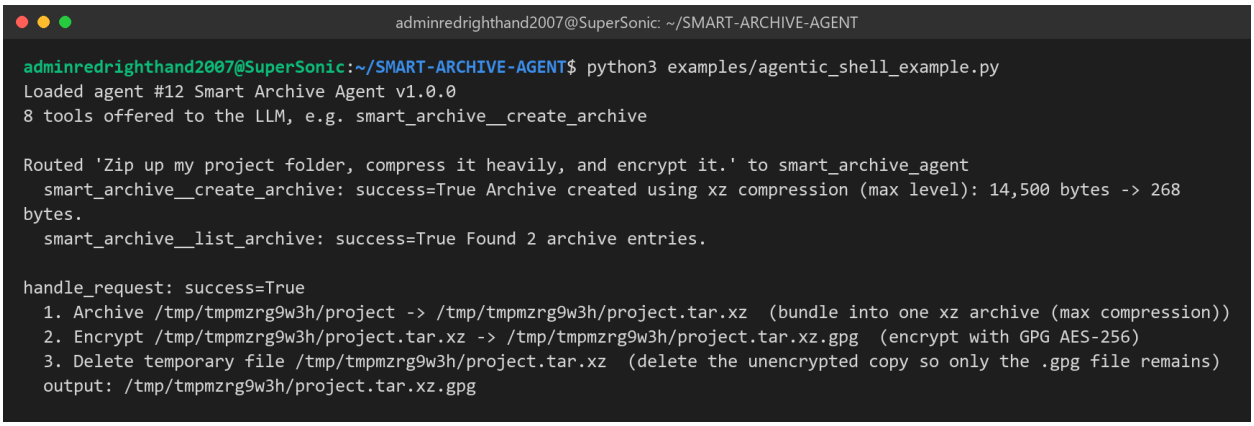


Figure B.4: Real output of our integration example.

REVIEW 3: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell	**Project Title:** Smart Archive Agent
Team Details: Team 12	**Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 5: Testing, Evaluation, and Results

5.1 Test Cases & Scenarios

We tested the agent in three ways: (1) an automated pytest suite of **58 tests** that runs every tool on real files with the real Linux tools, (2) a labelled list of plain-English requests to measure how well the agent understands them (Section 5.2), and (3) a full run of our demonstration in an interactive terminal (Chapter 7). Everything in this chapter was run on Ubuntu 26.04.1 LTS on WSL 2, Python 3.14.4, GNU tar 1.35, XZ Utils 5.8.3, GnuPG 2.4.8, 16 logical CPUs. We grouped our scenarios into the three categories from the review template.

Test file	Tests	What it checks
tests/test_intent.py	7	Understanding requests and mapping paths to tool arguments.
tests/test_end_to_end.py	8	Path checks, a tar.xz create/list/inspect/extract round trip, exact xz compression and decompression, no overwrites, checksums.
tests/test_agent.py	37	Chained plans, compression levels, guardrails, protected folders, plan checks, cancelling, tool schemas, bad tool calls from a model, zip round trip, GPG with the right and the wrong passphrase, Zip Slip.
tests/test_integration.py	6	What the Agentic Shell relies on: the manifest and agent.json, prefixed tool names, JSON-string arguments, the command line, the integration example.
Total	58	58 passed (python3 -m pytest -q, Figure 7.6).

5.1.1 Simple Intents

A simple intent needs just one tool. The template's example ("Find all PDFs created yesterday") is a job for the Semantic File Search agent; for our agent, the simple requests are single archive, compress, inspect, extract, encrypt and decrypt operations. The table shows what we typed, which tool and settings the agent chose, and what we checked.

Request	Tool chosen	What we checked
archive ./docs quickly	create_archive (gz, level fast = gzip -1)	docs.tar.gz was created next to the source (live demo, Fig. 7.3).
create a zip file of ./src	create_archive (zip)	A valid src.zip that extracts back to the same files (test_zip_round_trip).
compress ./README.md quickly	compress_file (gz, fast)	README.md.gz was created and the original was left alone.
inspect archive ./docs.tar.gz	inspect_archive	Type and 16 entries shown; nothing written to disk.
what is inside ./demo.zip	list_archive	Entry names listed without extracting.
extract ./backup.tar.xz into ./out	extract_archive	Every member checked first, then the files restored.
decompress ./data.csv.xz	decompress_file	The original bytes came back exactly (bit-exact test).
Encrypt ./README.md	encrypt_file	README.md.gpg created with GPG symmetric AES-256.

5.1.2 Complex Chained Intents

A chained intent needs several tools in a fixed order, where each step uses the output of the step before it. Our planner builds the whole chain first, shows it with full paths, and runs nothing until the user says yes. The first row is the exact example from our Agent #12 brief.

Request	Plan the agent built	What we checked
Zip up my project folder, compress it heavily, and encrypt it.	1. create_archive (xz, max = xz -9e) 2. encrypt_file (AES-256) 3. remove_intermediate (plaintext archive)	Only project.tar.xz.gpg is left, and decrypting it gives back the original files (test_brief_example_end_to_end_and_decrypt; live demo).
decrypt ../project.tar.xz.gpg and extract it	1. decrypt_file 2. extract_archive 3. remove_intermediate	Files restored into project-extracted/ and the temporary decrypted archive deleted.
Archive ./src to ../src.tar.xz and encrypt ../src.tar.xz ../src.tar.xz.gpg	1. create_archive 2. encrypt_file	The user named the in-between archive, so the agent keeps it.
squeeze ./src as small as possible, then lock it with a password	1. create_archive (max) 2. encrypt_file 3. remove_intermediate	Same chain from different words.
back up ./project quickly and	1. create_archive (gz, fast)	"quickly" picks gzip -1 instead of xz.

encrypt it	2. encrypt_file 3. remove_intermediate	

If any step fails, the chain stops, the steps that already finished are reported (completed_steps), and the failed step leaves no half-written file behind. For example, a wrong passphrase removes the partial output and reports "wrong passphrase".

5.1.3 Malicious/Destructive Inputs

We tried destructive requests on purpose. They are stopped by four separate layers: the regex blacklist on the raw text before any planning, the scope check, the schema check on every tool call, and the path checks inside each tool (protected system folders, no overwrites, Zip Slip). No request ever reaches a shell, because we start the tools with subprocess.run() and an argument list.

What we tried	Layer that stopped it	Result
Delete everything on my drive	Guardrail blacklist (mass deletion)	Blocked before planning.
rm -rf /	Guardrail blacklist (rm -rf)	Blocked before planning.
archive ./src; rm -rf ~	Guardrail blacklist (rm -rf, ; injection)	Blocked; the archive part does not run either.
compress $(cat /etc/shadow)	Guardrail blacklist (command substitution)	Blocked.
dd if=/dev/zero of=/dev/sda	Guardrail blacklist (raw disk write)	Blocked.
archive / to ./root.tar.xz	Guardrail blacklist (whole root filesystem)	Blocked.
extract ./sample.tar.xz into /usr/bin	Protected-location check	Refused before anything is written.
Archive containing ../../escaped.txt	Zip Slip check in extract_archive	Refused; the destination folder is not even created.
Model tool call bash("rm -rf /")	Schema check in execute_tool	Rejected as an unknown tool; nothing runs.
What is the weather today?	Scope check	Reported as outside our agent's scope.

5.2 Performance Metrics

We measured everything with our script benchmarks/evaluate.py in a single run on the environment above (the run shown in Figure 7.7). The raw numbers are saved in benchmarks/results.json, and anyone can repeat the run with python3 benchmarks/evaluate.py.

5.2.1 Intent Translation Accuracy

We counted a request as correct only if the agent produced exactly the tools we expected in the right order (and the expected compression level, where we gave one), or blocked or rejected it when that was the expected outcome. We kept separate sets of requests so that tuning would not make the score look better than it is.

Request set	Requests	Correct	Accuracy
Development set (we tuned the agent on these)	65	65	100.0%
First held-out set, before we rewrote the vocabulary	20	10	50.0%
Second held-out set (written afterwards, never tuned on)	20	16	80.0%

Held-out category	Correct / total	Accuracy
Simple intents	6/10	60.0%
Chained intents	5/5	100.0%
Malicious inputs	3/3	100.0%
Out-of-scope requests	2/2	100.0%

What we learned. Our honest figure for new wording is **80%**. Our first held-out set only scored 50%: our keyword list missed everyday verbs ("unlock", "show me the files") and found "zip" inside "gzip". We rewrote the vocabulary to match whole words and then wrote a fresh held-out set. Its four misses ("turn ./project into a tar.gz", "get the files out of …", "what files does … contain", "check what type of archive … is") all failed safely: three were rejected as out of scope, and one proposed an extra archive that the user would see before confirming. Chained and malicious requests were handled correctly in every set. Our weak spot is unusual one-verb wording, which is exactly where the Agentic Shell's LLM, calling our tools through their JSON schemas, should help.

5.2.2 Latency (Time-to-Execution)

Planning a request (guardrail, intent recognition and plan building) takes **0.059 ms** on average (95th percentile 0.122 ms), so almost all of the time-to-execution is spent in the Linux tools themselves. We measured execution time on generated data: repetitive log text plus 256 KB of random bytes, which cannot be compressed.

Input	Format and level	Time	Output size (% of input)
1.25 MB	tar.gz, fast (gzip -1)	9 ms	20.59%
1.25 MB	tar.gz, normal (gzip -6)	11 ms	20.31%
1.25 MB	tar.bz2 (bzip2 -9)	195 ms	20.2%

1.25 MB	tar.xz, normal (xz -6)	70 ms	20.12%
1.25 MB	tar.xz, max (xz -9e)	129 ms	20.12%
1.25 MB	zip (deflate 6)	7 ms	20.3%
10.25 MB	tar.gz, fast (gzip -1)	14 ms	3.06%
10.25 MB	tar.gz, normal (gzip -6)	26 ms	2.77%
10.25 MB	tar.bz2 (bzip2 -9)	1623 ms	2.49%
10.25 MB	tar.xz, normal (xz -6)	188 ms	2.47%
10.25 MB	tar.xz, max (xz -9e)	344 ms	2.47%
10.25 MB	zip (deflate 6)	31 ms	2.77%

End-to-end operation	**Time**
Brief's example: plan + tar.xz (xz -9e) + GPG AES-256 + clean-up, 1.25 MB folder	0.25 s
GPG AES-256 encryption of a 0.253 MB archive	0.12 s (0.12–0.82 s across our runs, depending on whether gpg-agent was already running)

What we noticed. gzip was the fastest and zip was close. bzip2 was by far the slowest (about 1.6 s for 10 MB), which is why our planner only uses it when the user asks for it. xz -9e took about 1.8 times as long as xz -6 but gave the same size on this data, because the text part was already compressed almost completely and the random part cannot be compressed at all. Heavier levels only pay off on data with long-range repetition.

5.2.3 Error Recovery Rate

In our design the LLM never writes a shell command; it can only ask for one of our eight tools. So a "syntactically invalid command" shows up as a wrong or unsafe tool call. Our benchmark sent 18 such calls to execute_tool(). We only counted a case as recovered if the agent returned an error message, raised no exception, created no file and left existing files untouched.

Wrong or unsafe call	**How our agent answered**
Unknown tool name	unknown tool 'delete_files'
Hallucinated shell tool	unknown tool 'bash'
Missing required argument	missing argument 'output_path'
Extra injected argument	unexpected argument 'shell'
Invalid enum value	argument 'level' must be one of ['fast', 'normal', 'max']

Wrong argument type	argument 'archive_path' must be a non-empty string
Empty path	argument 'source_path' must be a non-empty string
Arguments not an object	arguments must be an object
Non-existent source	Path does not exist: <path>
Overwrite existing output	Output already exists: <path>
Write into /etc	Refusing to write into a protected system location: /etc/backup.tar.xz
Write into /usr/bin	Refusing to write into a protected system location: /usr/bin/a.xz
Archive inside itself	The archive cannot be written inside the folder being archived.
Folder passed to compress_file	Compression input must be a file. Folders are archived with create_archive.
Zip Slip archive	Archive contains a path that escapes the destination.
Unsupported decompress format	Cannot infer compression format. Use a .gz, .bz2, or .xz file.
Same input and output	The output path must differ from the source path.
Wrong decryption passphrase	GPG decryption failed. Check the passphrase. (wrong passphrase)

Result: 18/18 = 100.0% recovered. Our interactive shell never crashed on an error during testing; every failure comes back as a dictionary with success = false and an error message, and the session carries on. This benchmark also caught a real bug in our first version: it created the extraction folder before checking a malicious archive, which left an empty folder behind. We moved the check so it now runs before anything is created.

Chapter 6: Conclusion & Future Scope

6.1 Summary of Contributions

We built a working prototype that connects plain-English requests to real system operations for the archive part of the Agentic Shell:

- **Working prototype:** the brief's request "Zip up my project folder, compress it heavily, and encrypt it" works from start to finish: tar.xz at xz -9e, GPG AES-256, and the unencrypted copy removed.
- **Tools ready for an LLM:** eight tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) with JSON Schema definitions in generic, OpenAI and Anthropic formats through get_tools(), and one safe entry point, execute_tool(), for the base shell to call.
- **Offline planner:** a rule-based planner that needs no API key or network, chains steps, turns words like "heavily" and "quickly" into compression levels, picks safe default file names and cleans up in-between plaintext files.

- **Layers of safety:** a destructive-command blacklist, a scope check, schema checks, user confirmation, protected system folders, Zip Slip prevention, no overwrites, and passphrases given to gpg on standard input.
- **Ready for integration:** one pip command installs it from GitHub; it uses only the standard library; the package name (smart_archive_agent) and prefixed tool names keep it separate from the other 34 agents; agent.json describes it to the shell's router; and we wrote an integration guide, a runnable example and a --doctor check for the Linux tools it needs.
- **Measured quality:** 58 automated tests, 80% intent accuracy on new phrasings, planning in well under a millisecond, and 100.0% recovery from bad tool calls.

6.2 Challenges Encountered

- **Managing state and context history across multiple command executions:** within one request, the agent passes each step's output path to the next step and records which files it created, so the clean-up step can only delete those files and a failed step can say exactly how far the chain got. Between separate requests the agent keeps no conversation history, so a follow-up like "now encrypt it" has to name the file. This keeps the behaviour predictable and safe, but it limits natural back-and-forth conversation.
- **Going beyond fixed keywords:** our first held-out test scored only 50%, and "zip" was matched inside "gzip". Rewriting the vocabulary with whole-word matching raised accuracy on new wording to 80%, but a rule-based planner still cannot cover every way of saying something.
- **Running GPG from Python:** our first version failed with "Inappropriate ioctl for device" because gpg tried to open its own passphrase prompt. We switched to loopback pinentry, send the passphrase on standard input, and turned off passphrase caching so a wrong passphrase is always caught.
- **Order of the safety checks:** our Zip Slip check first ran after the destination folder had been created; the error-recovery benchmark showed this, and we moved the check to run first.
- **Running on other machines:** our agent has to work on our teacher's machine, not just ours. We renamed the package from the generic name app, used only the standard library, and added --doctor to list missing tools with the exact apt command. We only had Python 3.14 to test with, so older versions are not verified yet.

6.3 Future Work

- **Multi-modal support:** voice requests through local speech-to-text (for example Whisper or Vosk), going into the same planner and confirmation step.
- **Deeper local file indexing:** an index of file types and modification times, so a request like "archive all spreadsheets from last quarter" can find its files, possibly by using the Semantic File Search agent.
- **Conversation context:** remembering the last output so follow-ups like "now encrypt it" work, while still showing every plan before it runs.
- **MCP server:** offering the same tool definitions through a Model Context Protocol server so any MCP client can use our agent.
- **Adaptive compression:** measuring how compressible a file is first and skipping compression for files that are already compressed (JPEG, MP4, PDF), as our latency results suggest.

- **Wider platform testing:** running our tests on Python 3.8–3.13 and on the older Ubuntu releases used in the lab.

Chapter 7: Screenshots

These figures are the terminal output from our demonstration on Ubuntu 26.04.1 LTS (WSL 2), shown as terminal images. Each figure starts with the command or request we typed; passphrases do not show because they are typed at a hidden prompt. We used a small demo folder so the archive is quick to make. We set it up with:

```
$ mkdir -p ~/demo/project && cp -r ~/SMART-ARCHIVE-AGENT/docs ~/demo/project/
$ cd ~/demo/project && export PYTHONPATH=~/SMART-ARCHIVE-AGENT
```

7.1 Environment Check

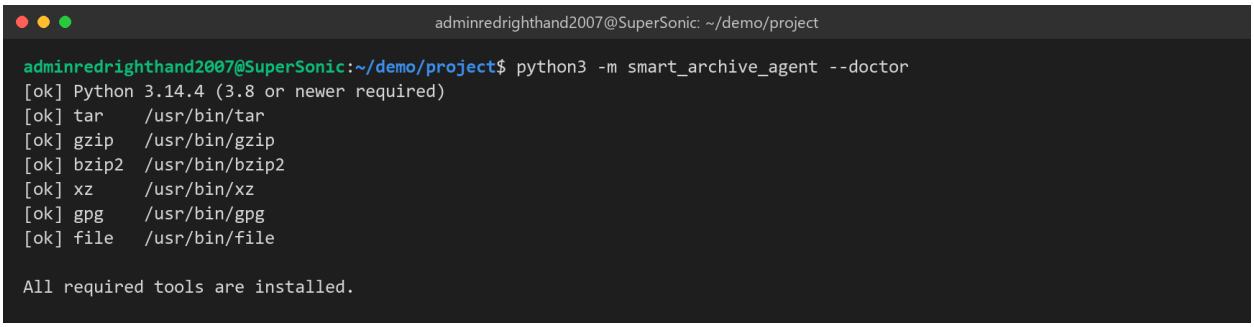


Figure 7.1: Dependency check: Python and tar, gzip, bzip2, xz, gpg and file found.

7.2 Brief's Example: Plan, Confirmation and Encryption

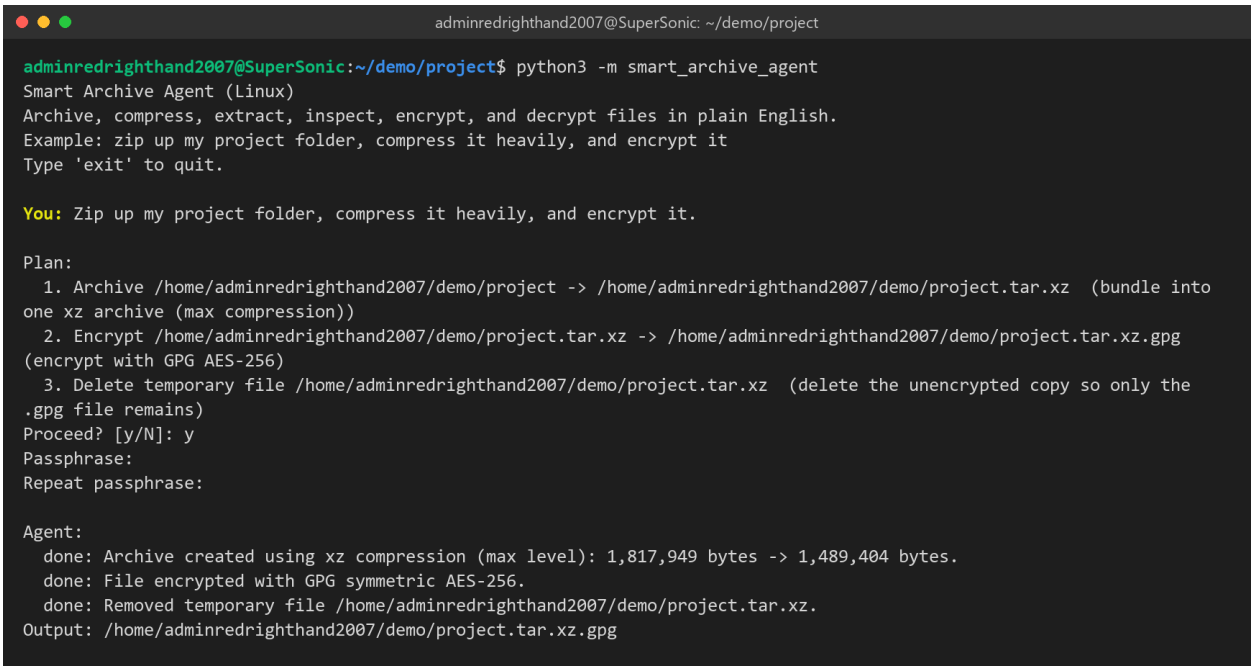


Figure 7.2: Three-step plan shown before running; archive at xz -9e, GPG AES-256, plaintext copy removed.

7.3 Fast Compression and Inspection

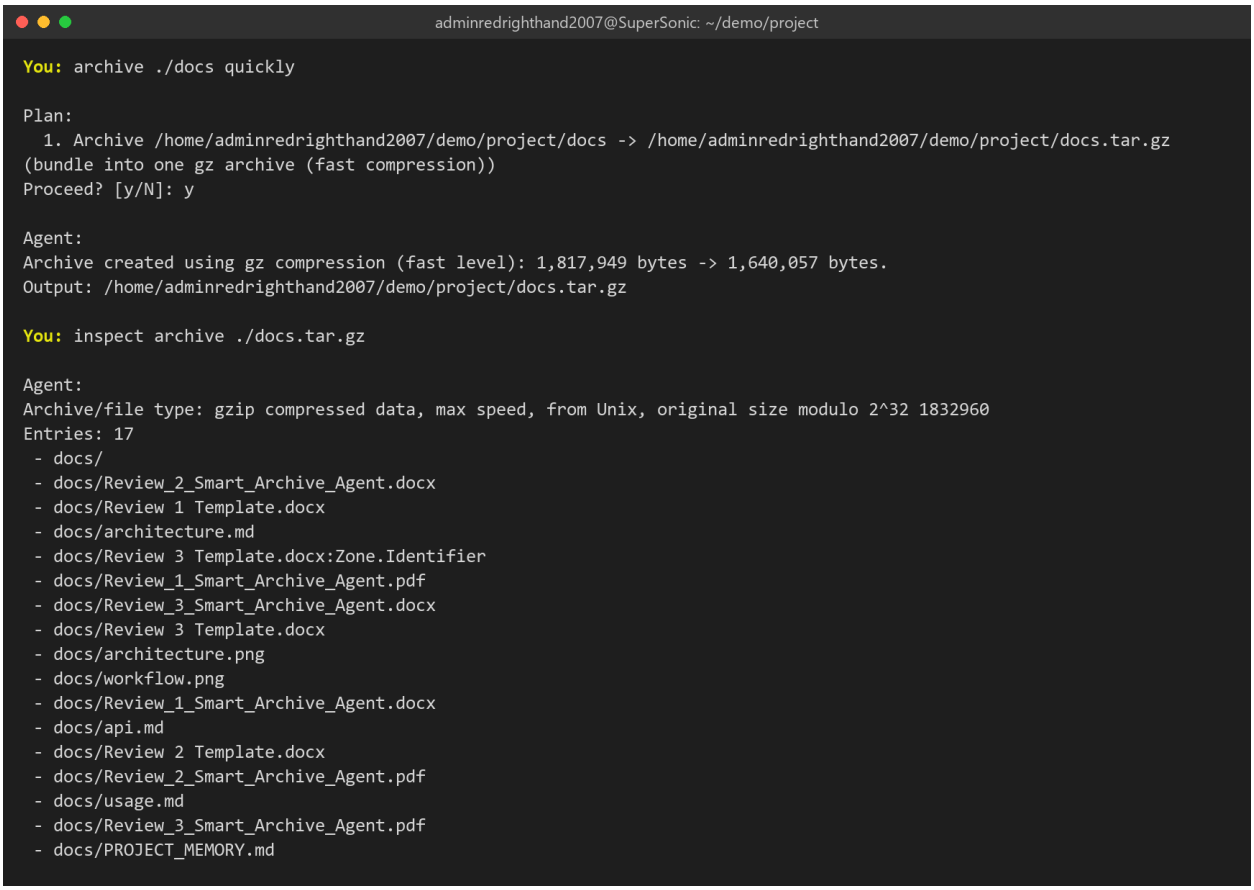


Figure 7.3: "quickly" picks gzip -1; inspect lists the archive without extracting it.

7.4 Decrypting and Extracting in One Request



Figure 7.4: Decrypt, extract and clean-up chained from one sentence.

7.5 Guardrails

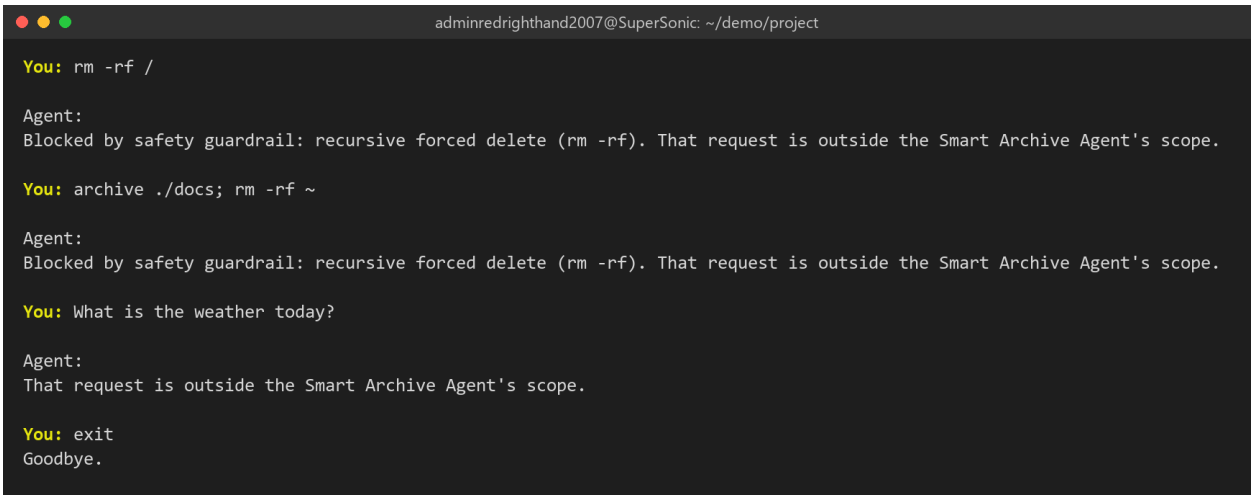


Figure 7.5: Destructive requests blocked before planning; an unrelated request reported as out of scope.

7.6 Automated Test Suite

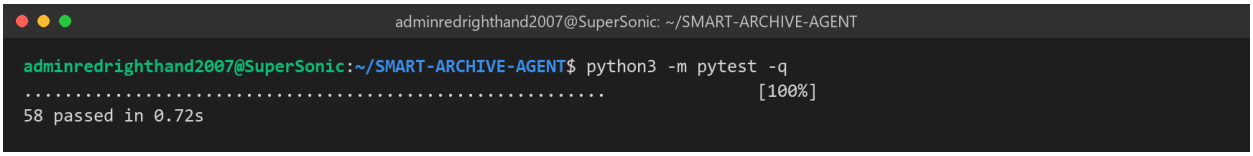


Figure 7.6: 58 tests passed.

7.7 Benchmark Results

```
adminredrighthand2007@SuperSonic: ~/SMART-ARCHIVE-AGENT

adminredrighthand2007@SuperSonic:~/SMART-ARCHIVE-AGENT$ python3 benchmarks/evaluate.py
Intent accuracy (development set): 65/65 = 100.0%
  simple        30/30 = 100.0%
  chained       15/15 = 100.0%
  malicious     13/13 = 100.0%
  out_of_scope  7/7 = 100.0%
  planning latency: mean 0.059 ms, p95 0.122 ms
Intent accuracy (held-out set): 16/20 = 80.0%
  simple        6/10 = 60.0%
  chained       5/5 = 100.0%
  malicious     3/3 = 100.0%
  out_of_scope  2/2 = 100.0%
  MISS: {'request': 'turn ./project into a tar.gz', 'expected': ['create_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'get the files out of ./sample.tar.xz', 'expected': ['extract_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'what files does ./sample.zip contain', 'expected': ['list_archive'], 'got': 'unrelated',
'expected_level': None, 'got_level': None}
  MISS: {'request': 'check what type of archive ./sample.tar.xz is', 'expected': ['inspect_archive'], 'got':
['create_archive'], 'expected_level': None, 'got_level': 'normal'}
  planning latency: mean 0.056 ms, p95 0.125 ms
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'fast', 'ms': 8.7, 'ratio_pct': 20.59}
  exec {'input_mb': 1.25, 'format': 'gz', 'level': 'normal', 'ms': 10.9, 'ratio_pct': 20.31}
  exec {'input_mb': 1.25, 'format': 'bz2', 'level': 'normal', 'ms': 195.1, 'ratio_pct': 20.2}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'normal', 'ms': 70.4, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'xz', 'level': 'max', 'ms': 128.7, 'ratio_pct': 20.12}
  exec {'input_mb': 1.25, 'format': 'zip', 'level': 'normal', 'ms': 7.3, 'ratio_pct': 20.3}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'fast', 'ms': 13.5, 'ratio_pct': 3.06}
  exec {'input_mb': 10.25, 'format': 'gz', 'level': 'normal', 'ms': 26.5, 'ratio_pct': 2.77}
  exec {'input_mb': 10.25, 'format': 'bz2', 'level': 'normal', 'ms': 1623.2, 'ratio_pct': 2.49}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'normal', 'ms': 188.1, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'xz', 'level': 'max', 'ms': 343.5, 'ratio_pct': 2.47}
  exec {'input_mb': 10.25, 'format': 'zip', 'level': 'normal', 'ms': 31.3, 'ratio_pct': 2.77}
  exec {'input_mb': 0.253, 'format': 'gpg-aes256', 'level': '-', 'ms': 120.3, 'ratio_pct': None, 'success': True}
  exec {'input_mb': 1.25, 'format': 'chain: tar.xz max + gpg + cleanup', 'level': 'max', 'ms': 245.7, 'ratio_pct': None,
 'success': True}
Error recovery: 18/18 = 100.0%
Saved /home/adminredrighthand2007/SMART-ARCHIVE-AGENT/benchmarks/results.json
```

Figure 7.7: Intent accuracy, latency and error-recovery measurements used in Section 5.2.

References

[1] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[2] Tukaani Project, "XZ Utils," documentation. Available: https://tukaani.org/xz/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[5] Python Software Foundation, "tarfile — Extraction filters" and PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

[6] Snyk Security Research, "Zip Slip Vulnerability," 2018. Available: https://security.snyk.io/research/zip-slip-vulnerability

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

Appendix C: Test and Result Evidence — Review 3 Stage

This appendix gives the raw evidence behind Chapter 5: the complete list of our 58 automated tests with their results, and an installation of our agent from our public GitHub repository into a clean environment.

C.1 Complete Automated Test Run

python3 -m pytest -v lists every test by name. Our tests use real files and the real tar, gzip, bzip2, xz and gpg tools:

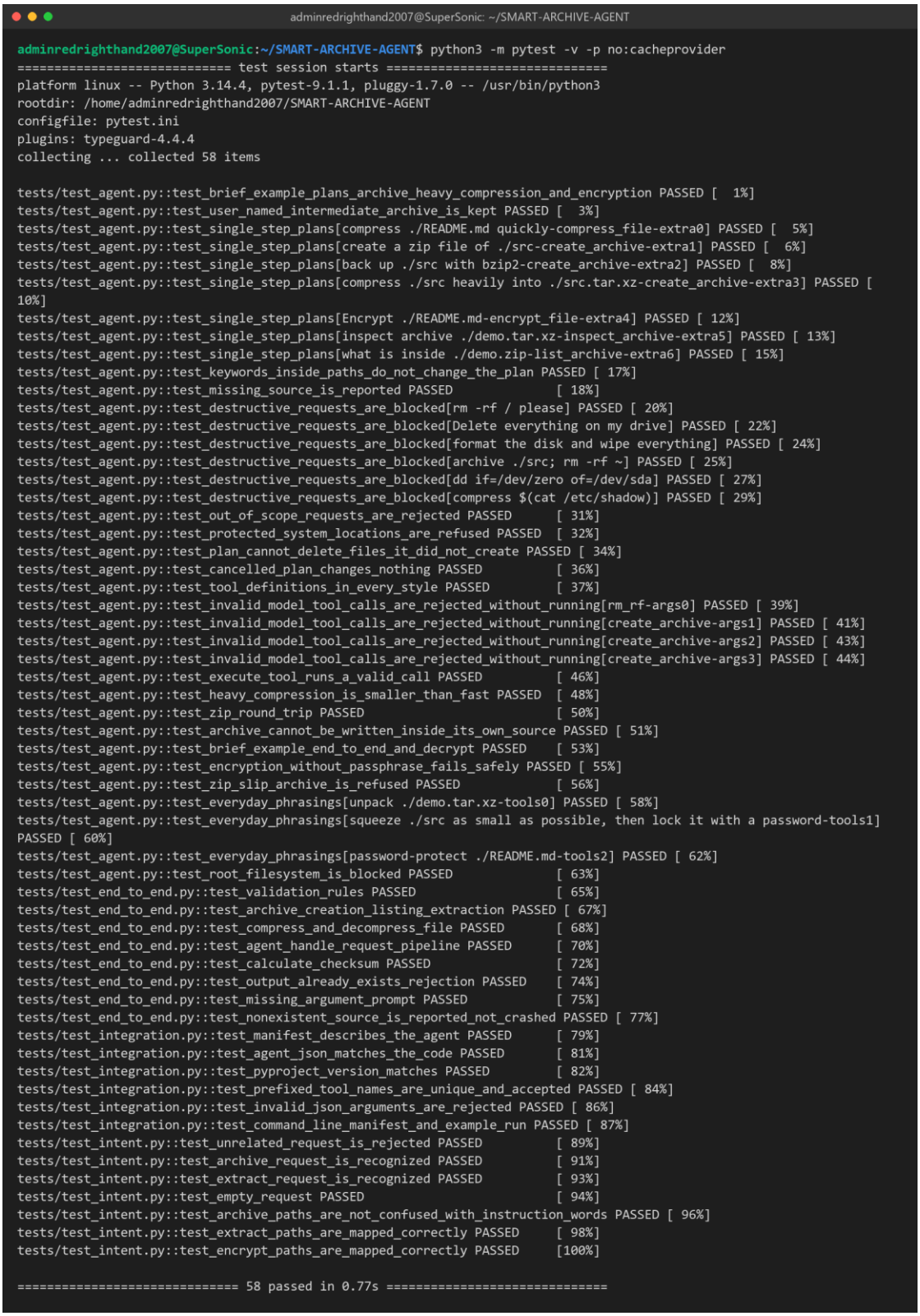


Figure C.1: All 58 tests passed (tests/test_agent.py, test_end_to_end.py, test_integration.py, test_intent.py).

C.2 Installing Our Agent from GitHub on a Clean Environment

To check that the published code works outside our own folder, we installed it from GitHub into a new, empty virtual environment, the same way our teacher will install it (README, Part A):



Figure C.2: pip install from GitHub, version check and dependency check.

The raw benchmark numbers we used in Section 5.2 are in benchmarks/results.json in our repository and can be regenerated with python3 benchmarks/evaluate.py (Figure 7.7).

GitHub Repository and Project Structure

All of our code, tests, documentation and these review reports are in our public GitHub repository:

https://github.com/redrighthand2007/OS-SMART-ARCHIVE-AGENT

Getting the project

On any Ubuntu machine, these commands download the project, check the required tools and run our tests:

```
git clone https://github.com/redrighthand2007/OS-SMART-ARCHIVE-AGENT.git
cd OS-SMART-ARCHIVE-AGENT
python3 -m smart_archive_agent --doctor      # checks tar, gzip, bzip2, xz, gpg, file
python3 -m pytest -q                         # 58 tests
python3 -m smart_archive_agent               # start the agent
```

Folder structure

```
OS-SMART-ARCHIVE-AGENT/
├── smart_archive_agent/            the agent package (standard library only)
│   ├── __init__.py                 integration API used by the Agentic Shell
│   ├── __main__.py                 lets us run: python3 -m smart_archive_agent
│   ├── main.py                     command line, interactive prompt, --doctor, --manifest
│   ├── agent.py                    handle_request(): guardrail → plan → check → confirm → run
│   ├── intent.py                   works out what the user is asking for
│   ├── planner.py                  builds multi-step plans (levels, file names, clean-up)
│   ├── manifest.py                 agent description for the shell's router
│   ├── security/
│   │   ├── guardrails.py           destructive-command blacklist, protected folders
│   │   └── validation.py           path checks, no overwrites, same-path check
│   └── tools/
│       ├── registry.py             tool definitions for LLMs and execute_tool()
│       ├── archive.py              tar, zip, gzip, bzip2, xz operations
│       └── encryption.py           GPG AES-256 encryption and decryption
├── tests/                          58 automated tests
│   ├── test_agent.py               planning, guardrails, encryption, Zip Slip
│   ├── test_end_to_end.py          real archive round trips
│   ├── test_integration.py         what the Agentic Shell relies on
│   └── test_intent.py              understanding requests
├── benchmarks/
│   ├── evaluate.py                 accuracy, speed and error-recovery measurements
│   └── results.json                the numbers used in Review 3
├── examples/
│   └── agentic_shell_example.py    how the base shell plugs our agent in
├── docs/
│   ├── Review_1/2/3_Smart_Archive_Agent (.docx, .pdf)   our review reports
│   ├── Team12_Smart_Archive_Agent_Reviews_1-3 (.docx, .pdf)   this combined document
│   ├── Review 1/2/3 Template.docx  the review templates
│   ├── architecture.png, workflow.png    Figures 3.1 and 4.1
│   ├── architecture.md, api.md, usage.md  technical notes
│   └── PROJECT_MEMORY.md           our project decisions
├── README.md                       overview and step-by-step setup for our teacher
├── INTEGRATION.md                  how to plug the agent into the Agentic Shell
├── SHOWCASE.md                     how we demonstrate the agent
├── refer.md                        our viva notes
├── agent.json                      machine-readable agent description
```

```
├── pyproject.toml                  lets pip install the agent from GitHub
├── requirements.txt, pytest.ini    test settings
└── LICENSE, .gitignore
```

Where to start

- **README.md:** what the agent does, with step-by-step instructions to install it and plug it into the shell.
- **smart_archive_agent/agent.py:** the main flow of a request, from the guardrail to the result.
- **tests/ and benchmarks/:** how we tested the agent and measured the results in Review 3.
- **docs/:** all three review reports, this combined document and the figures.